

Guia de Arquitetura, Quality Gates & DevSecOps

Especificação técnica institucional para projetos **Java 21 LTS** (Spring Boot 3.3) e **Angular 22 Zoneless**, integrados a esteiras de CI/CD no GitHub Actions com **SonarQube**, **Semgrep**, **Betterleaks**, **Syft/Grype**, **Nuclei** e **OWASP ZAP**.

Java 21 LTS

Spring Boot 3.3

Angular 22 (Zoneless)

Vitest 2.x

SonarQube (Sonar way)

Semgrep SAST

Betterleaks

Syft & Grype

DAST: Nuclei & ZAP

Template Oficial Pronto para Uso

Faça o download do projeto base Maven multi-módulo já configurado com todos os workflows.

[Baixar fullstack-starter.zip](#)

00 Template de Scaffolding & Repositórios

Cada equipe deve inicializar e versionar o projeto utilizando **estritamente** as seguintes nomenclaturas padronizadas no GitHub:

- **Equipe DaMatch:** `senac-fullstack-damatch`
- **Equipe Azrael:** `senac-fullstack-azrael`

0.1 Estrutura do Repositório Gerado

FULLSTACK-STARTER/ — ÁRVORE DE ARQUIVOS

```
fullstack-starter/
├── .github/
│   └── workflows/
│       └── ci-cd.yml                <-- Pipeline (Lint -> Testes -> SAST/Betterleaks/Sy
├── backend/
│   ├── checkstyle.xml              <-- Regras de linting Java
│   ├── pom.xml                    <-- Java 21 LTS, Spring Boot 3.3, JaCoCo, ArchUnit,
│   └── src/
│       ├── main/java/br/senac/fullstack/
│       │   ├── FullstackApplication.java
│       │   ├── config/CorsConfig.java
│       │   ├── controller/HealthCheckController.java
│       │   ├── dto/HealthStatusDTO.java
│       │   └── service/HealthCheckService.java
│       └── test/java/br/senac/fullstack/
│           ├── FullstackApplicationTests.java
│           ├── controller/HealthCheckControllerTest.java
│           ├── service/HealthCheckServiceTest.java
│           └── architecture/ArchitectureRulesTest.java
├── frontend/
│   ├── eslint.config.mjs          <-- Regras de linting TypeScript / Angular
│   ├── package.json              <-- Angular 22 (Zoneless, Signals), Vitest, ESLint
│   ├── angular.json              <-- Configuração de build moderna (@angular/build)
│   ├── tsconfig.json
│   ├── vite.config.mts           <-- Configuração do runner moderno Vitest
│   └── src/app/
│       ├── app.component.ts      <-- Standalone + Signals + OnPush
│       ├── app.component.html    <-- Control Flow moderno (@if / @else)
│       ├── app.component.css
│       └── app.component.spec.ts  <-- Testes unitários com Vitest
├── pom.xml                       <-- Orquestrador Maven raiz + sonar-maven-plugin
├── README.md                     <-- Documentação de arquitetura e comandos locais
└── GITHUB_SETUP_GUIDE.md        <-- Guia passo a passo de setup e secrets
```

01 Restrições Arquiteturais & de Código

1.1 Arquitetura em Camadas & ArchUnit

Fluxo de dependência obrigatório: `Controller` → `Service` → `Repository` → `Domain`, com DTOs cruzando apenas a fronteira do `Controller`.

#	REGRA ARQUITETURAL	JUSTIFICATIVA
R1	<code>Controller</code> nunca injeta <code>Repository</code> diretamente	Evita acoplamento direto à persistência nos testes web
R2	<code>Service</code> nunca conhece tipos de <code>jakarta.servlet</code> ou anotações web	Lógica de negócio deve ser testável sem contexto HTTP
R3	<code>Repository</code> não contém lógica de negócio	<code>Repository</code> é infraestrutura de persistência
R4	Pacotes organizados por <i>feature</i>	Evita a camada Deus; <code>br.senac.fullstack.{feature}.*</code>
R5	Validação estrita em tempo de build com ArchUnit	Impede regressões arquiteturais de dependências invertidas

ARCHITECTURERULESTEST.JAVA

```
@AnalyzeClasses(packages = "br.senac.fullstack", importOptions = ImportOption.DoNotIncludeClasses)
public class ArchitectureRulesTest {

    @ArchTest
    static final ArchRule controllerClassesShouldBeAnnotatedWithRestController =
        classes().that().resideInAPackage("..controller..")
            .should().beAnnotatedWith(RestController.class);

    @ArchTest
    static final ArchRule serviceClassesShouldBeAnnotatedWithService =
        classes().that().resideInAPackage("..service..")
            .should().beAnnotatedWith(Service.class);

    @ArchTest
    static final ArchRule layeredArchitectureRule = layeredArchitecture()
        .consideringOnlyDependenciesInLayers()
        .layer("Controller").definedBy("..controller..")
        .layer("Service").definedBy("..service..")
        .whereLayer("Controller").mayNotBeAccessedByAnyLayer()
        .whereLayer("Service").mayOnlyBeAccessedByLayers("Controller");
}
```

1.2 DTOs com Java 21 Records & Mappers

REGRA ABSOLUTA

Nenhuma entidade JPA cruza a fronteira do Controller. Utilize Records imutáveis para DTOs com anotações de validação do Bean Validation.

HEALTHSTATUSDTO.JAVA

```
public record HealthStatusDTO(  
    @NotBlank String status,  
    @NotBlank String application,  
    @NotBlank String environment,  
    @NotNull Instant timestamp,  
    @PositiveOrZero long uptimeMillis  
) {}
```

1.3 Tratamento Global de Exceções — RFC 7807

Uso obrigatório do `ProblemDetail` nativo do Spring 6/3.x. É estritamente proibido retornar mensagens internas de erro não sanitizadas ou stacktraces ao cliente.

1.4 Autenticação & Identidade — LLDAP SSO

O ecossistema utiliza autenticação centralizada no servidor LLDAP (<https://lldap.fchicout.dev>). Os estudantes autenticam utilizando seu usuário de e-mail (ex: `christopher.carvalho`) com permissões mapeadas via grupos LDAP (`damatch_admin` e `azrael_admin`).

1.5 Frontend Moderno: Angular 22 Zoneless, Signals & OnPush

O frontend opera na vanguarda da arquitetura do Angular 22:

- **Zoneless (Sem zone.js):** Inicialização e reatividade acionadas diretamente por Signals reativos (`provideZonelessChangeDetection()`).
- **Change Detection OnPush:** Avaliação cirúrgica de componentes no DOM.
- **Control Flow Moderno:** Diretivas legadas substituídas por `@if`, `@else` e `@for` nativos.

02 Quality Gates no SonarQube & Testes Automatizados

O servidor sonar.fchicout.dev adota o Quality Gate oficial e padrão "**Sonar way**" (baseado no princípio *Clean as You Code*). O pipeline do GitHub Actions valida e bloqueia o build (-Dsonar.qualitygate.wait=true) caso qualquer uma das seguintes regras seja violada no código novo:

MÉTRICA (CHAVE INTERNA)	OPERADOR	CONDIÇÃO DE REPROVAÇÃO	LIMITE OBRIGATÓRIO
Coverage on New Code (new_coverage)	LT (Menor que)	Menor que 80.0%	≥ 80.0%
Duplicated Lines on New Code (new_duplicated_lines_density)	GT (Maior que)	Maior que 3.0%	≤ 3.0%
Security Hotspots Reviewed (new_security_hotspots_reviewed)	LT (Menor que)	Menor que 100.0%	100.0%
New Issues / Violations (new_violations)	GT (Maior que)	Maior que 0	0 Novas Violações

CLEAN AS YOU CODE

O **Sonar way** garante que todo novo commit ou Pull Request mantenha dívida técnica zero: **zero** novos bugs, **zero** novas vulnerabilidades, **zero** novos code smells, e no mínimo **80%** de cobertura por testes automatizados.

2.2 Padrões de Teste: JUnit 5, JaCoCo & Vitest

Backend: JaCoCo gera o relatório XML em `backend/target/site/jacoco/jacoco.xml` a cada execução de `mvn test`.

Frontend: O runner oficial **Vitest 2.x** executa os testes em milissegundos sem abrir navegadores pesados:

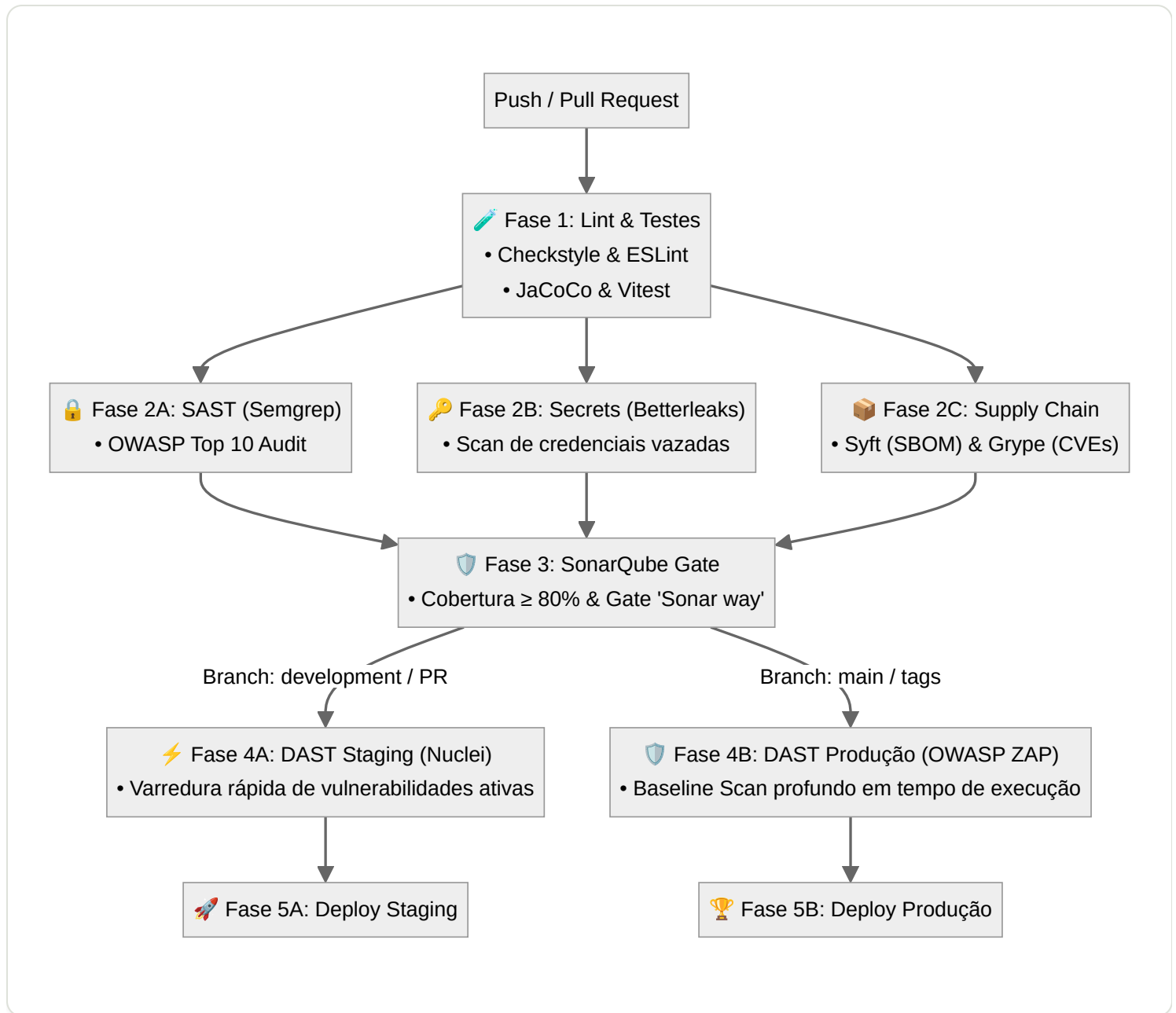
VITE.CONFIG.MTS

```
import { defineConfig } from 'vite';
import angular from '@analogjs/vite-plugin-angular';

export default defineConfig({
  plugins: [angular()],
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: ['src/test-setup.ts'],
    include: ['src/**/*.spec.ts'],
    coverage: { provider: 'v8', reporter: ['text', 'lcov', 'html'] }
  }
});
```

03 DevSecOps: Pipeline de Segurança Contínua

A esteira de integração contínua implementa uma blindagem completa em 5 camadas (Shift-Left Security):



3.1 Linting Obrigatório (Checkstyle & ESLint)

Verificação estática de código limpo e padrões arquiteturais executada antes de qualquer teste.

3.2 SAST: Auditoria Estática com Semgrep

Analisa o código em busca de injeções de SQL/Comando, controle de acesso quebrado, má configuração de headers e antipadrões de segurança do Spring e TypeScript.

3.3 Secret Scanning com Betterleaks

Utiliza o **Betterleaks** para varrer todo o histórico do Git e arquivos alterados, detectando tokens de API, senhas e certificados privados antes do merge.

3.4 Supply Chain Security: Syft (SBOM) & Grype (CVEs)

O **Syft** gera o inventário de software padronizado (SBOM SPDX-JSON) e o **Grype** cruza as dependências contra vulnerabilidades conhecidas em tempo de compilação.

3.5 DAST Diferenciado por Ambiente: Nuclei vs OWASP ZAP

- **Ambiente de Staging (Branch development):** Varredura rápida orientada a templates com **ProjectDiscovery Nuclei** para feedback ágil aos desenvolvedores.
- **Ambiente de Produção (Branch main / Tags):** Varredura profunda com **OWASP ZAP Baseline Scan** para homologação de releases.

04 Governança Git, Política de Branches & Happy Path

4.1 Política de Branches

● **main (Produção)**

- **100% Protegida:** Sem push direto
- Recebe merges exclusivamente de `development` ao fechar uma versão
- Dispara DAST (OWASP ZAP) e Deploy em Produção

● **development (Staging)**

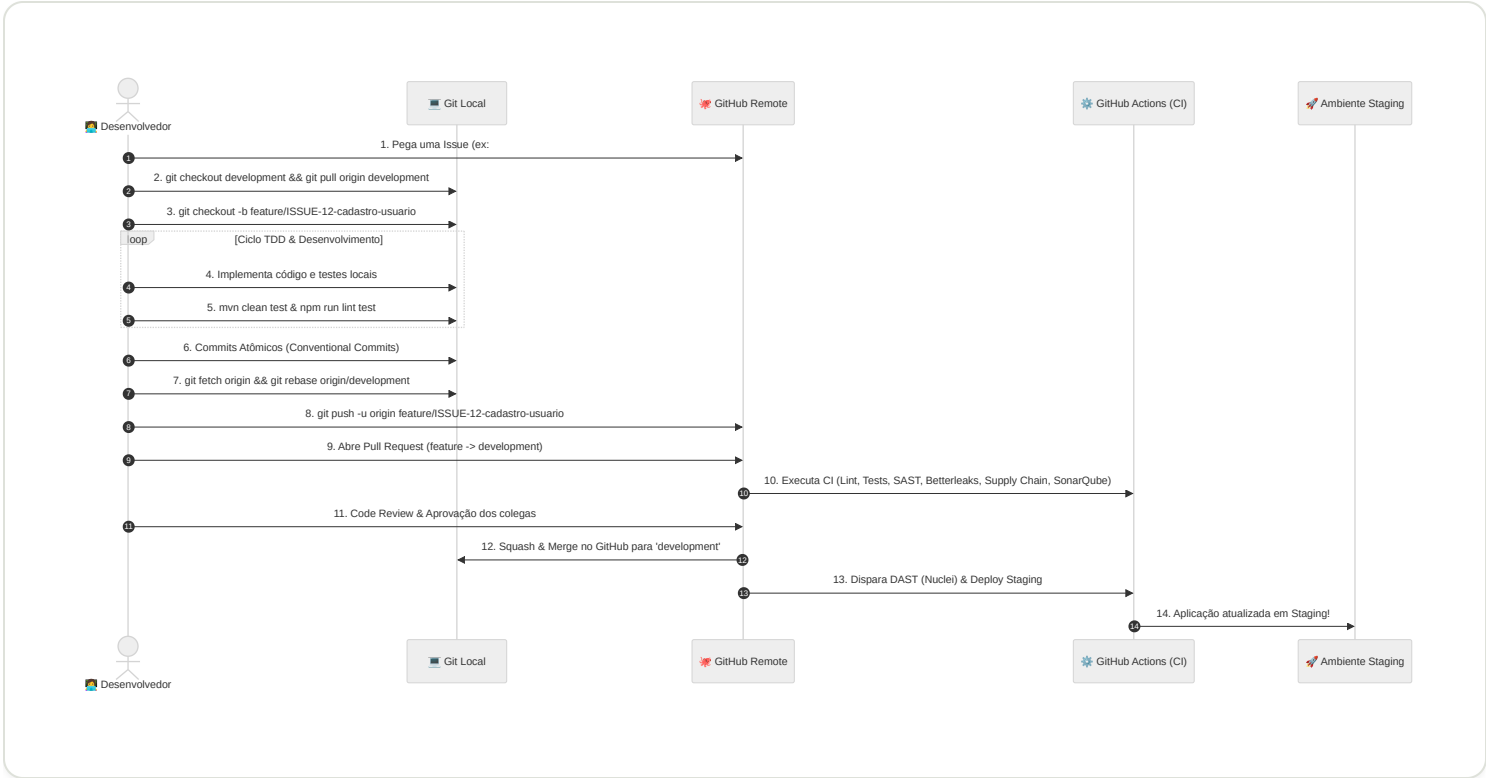
- Branch base de integração do time
- Entrada exclusiva via Pull Request aprovado
- Dispara DAST (Nuclei) e Deploy em Staging

● **feature/* · bugfix/***

- Ramificadas a partir da `development`
- Uma branch por Issue (ex: `feature/ISSUE-12-auth`)
- Descartáveis após o Squash & Merge

4.2 O Caminho Feliz (Happy Path)

Ciclo de vida operacional padrão de um desenvolvedor desde o recebimento da Issue até a entrega em Staging:



4.3 Commits Convencionais & SemVer

PREFIXO	SIGNIFICADO	IMPACTO SEMVER
feat(...)	Nova funcionalidade adicionada	MINOR 1.2.0 → 1.3.0
fix(...)	Correção de bug	PATCH 1.2.0 → 1.2.1
refactor(...)	Refatoração sem alterar comportamento	— Sem release sozinho
test(...)	Adição ou ajuste de testes	— Sem release sozinho
BREAKING CHANGE:	Quebra de compatibilidade externa	MAJOR 1.2.0 → 2.0.0

05 Configuração Multi-Profile & Infraestrutura

Estrutura: application.yml (comum) + application-{test,staging,prod}.yml.

PROFILE	BANCO DE DADOS	DDL-AUTO	OBSERVABILIDADE
test	jdbc:sqlite::memory:	create-drop	DEBUG completo
staging	/data/staging/extensao.db	validate	INFO + métricas
prod	/data/prod/extensao.db	validate	WARN, apenas health

APPLICATION-PROD.YML

```
spring:
  datasource:
    url: jdbc:sqlite:/data/prod/extensao.db
    hikari:
      maximum-pool-size: 1
  jpa:
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true
  ldap:
    urls: ${LDAP_PROD_URL:ldap://lldap.homelab.local:3890}
    password: ${LDAP_ADMIN_PASSWORD}
  logging:
    level:
      root: WARN
  management:
    endpoint:
      health:
        show-details: never
```